

go-zkfp Development Guide

Go Binding for ZKFinger Reader SDK (libzkfp.dll)

Based on ZKTeco ZKFinger Reader SDK C API Version 2.0

Contents

1	Overview	3
2	Disclaimer	3
3	System Requirements	3
4	Installation and Deployment	3
5	Description of Go Interfaces	3
5.1	Design Conventions	3
5.1.1	Function Mapping	4
5.2	Type Definition	4
5.2.1	Constants	5
5.3	Interface Description	5
5.3.1	SetDLLPath	5
5.3.2	Init	5
5.3.3	Terminate	5
5.3.4	GetDeviceCount	6
5.3.5	OpenDevice	6
5.3.6	Device.Close	6
5.3.7	Device.SetParameter	6
5.3.8	Device.GetParameter	7
5.3.9	Device.AcquireFingerprint	7
5.3.10	Device.AcquireFingerprintImage	7
5.3.11	NewDB	7
5.3.12	DB.Close	8
5.3.13	DB.Merge	8
5.3.14	DB.Add	8
5.3.15	DB.Delete	8
5.3.16	DB.Clear	9
5.3.17	DB.Count	9
5.3.18	DB.Identify	9
5.3.19	DB.Match	9
5.3.20	DB.ExtractFromImage	9
6	Usage Example	10

7	Appendixes	11
7.1	Appendix 1: List of Common Parameter Codes	11
7.2	Appendix 2: Descriptions of Returned Error Values	11
7.3	Appendix 3: Notes for Implementers	11

1 Overview

`go-zkfp` is a Go binding for `libzkfp.dll`, the ZKFinger Reader SDK from ZKTeco. It lets Go programs enumerate ZKTeco fingerprint readers, capture images and templates, and register, identify and match fingerprints. This document describes the Go API. Each Go function maps one-to-one to a C function of the ZKFinger Reader SDK (C API Version 2.0).

2 Disclaimer

`go-zkfp` is an unofficial binding and does not include the ZKTeco SDK. `libzkfp.dll` must be obtained by installing the ZKFinger SDK from ZKTeco, and its use is subject to the ZKTeco license: you shall not use, copy, modify, lease, or transfer any part of the SDK beyond the clauses of the original SDK document.

3 System Requirements

1. Operating system: Windows XP or a later version
2. Go 1.18 or later
3. ZKFinger SDK 5.x / ZKOnline SDK 5.x installed (provides `libzkfp.dll` and the reader driver)
4. The architecture of the Go program (`GOARCH=386` or `amd64`) must match the architecture of `libzkfp.dll`
5. `cgo` and a C compiler are **not** required: the library is loaded at runtime with `golang.org/x/sys/windows` (`windows.NewLazyDLL`), whose calls use the `stdcall` convention that the SDK requires

4 Installation and Deployment

1. Install ZKFinger SDK 5.x / ZKOnline SDK 5.x.
2. Add the package to your module:

```
go get github.com/elmyrockers/go-zkfp
```
3. Make `libzkfp.dll` loadable: place it next to your executable, or in a directory on `PATH`. To load it from a custom location call `zkfp.SetDLLPath` **before** any other function.
4. Import the package:

```
import zkfp "github.com/elmyrockers/go-zkfp"
```

5 Description of Go Interfaces

5.1 Design Conventions

- **Errors.** Instead of returning an integer code, functions return a Go **error**. A non-zero SDK result is returned as a value of type `zkfp.Error` (see Appendix 2), so it can be compared with `errors.Is` or `errors.As`.
- **Handles.** The C `HANDLE` types are wrapped in two Go types: `*Device` (reader instance) and `*DB` (algorithm cache). Functions that take a handle in C are methods in Go.

- **Buffers.** The C pattern of “pointer + size” pairs becomes a Go []byte. Output buffers are allocated by the binding and returned already trimmed to the actual returned size.
- **Cleanup.** Close methods are idempotent: calling them more than once is safe and returns nil.
- **Concurrency.** The SDK is not documented as thread-safe. A *Device or *DB must not be used from several goroutines at the same time unless the caller serializes access. The binding does not add locks of its own.

5.1.1 Function Mapping

C function	Go function / method
ZKFPM_Init	zkfp.Init
ZKFPM_Terminate	zkfp.Terminate
ZKFPM_GetDeviceCount	zkfp.GetDeviceCount
ZKFPM_OpenDevice	zkfp.OpenDevice
ZKFPM_CloseDevice	(*Device).Close
ZKFPM_SetParameters	(*Device).SetParameter
ZKFPM_GetParameters	(*Device).GetParameter
ZKFPM_AcquireFingerprint	(*Device).AcquireFingerprint
ZKFPM_AcquireFingerprintImage	(*Device).AcquireFingerprintImage
ZKFPM_DBInit	zkfp.NewDB
ZKFPM_DBFree	(*DB).Close
ZKFPM_DBMerge	(*DB).Merge
ZKFPM_DBAdd	(*DB).Add
ZKFPM_DBDel	(*DB).Delete
ZKFPM_DBClear	(*DB).Clear
ZKFPM_DBCount	(*DB).Count
ZKFPM_DBIdentify	(*DB).Identify
ZKFPM_DBMatch	(*DB).Match
ZKFPM_ExtractFromImage	(*DB).ExtractFromImage

5.2 Type Definition

```
package zkfp

// Device is an opened fingerprint reader (wraps the C device HANDLE).
type Device struct { /* unexported */ }

// DB is an algorithm cache (wraps the C hDBCACHE HANDLE).
type DB struct { /* unexported */ }

// Error is a non-zero result code returned by libzkfp.dll.
type Error int32

func (e Error) Error() string // e.g. "zkfp: no device connected (-3)"

// Format selects the template format (parameter code 10001).
type Format int32

const (
```

```

FormatANSI378 Format = 0 // ANSI INCITS 378
FormatISO      Format = 1 // ISO/IEC 19794-2
)

```

5.2.1 Constants

1. Maximum length of a template
[Definition] `const MaxTemplateSize = 2048`
2. Fingerprint 1:1 threshold parameter code
[Definition] `const ParamThreshold = 1`
3. Fingerprint 1:N threshold parameter code
[Definition] `const ParamMThreshold = 2`

The device parameter codes of Appendix 1 are exported as `Param...` constants, and the error codes of Appendix 2 as `Err...` values.

5.3 Interface Description

5.3.1 SetDLLPath

[Function] `func SetDLLPath(path string)`

[Purpose] This function is used to choose the location of `libzkfp.dll`. It has no C counterpart.

[Parameter Description] `path` - Full path of `libzkfp.dll`

[Note] It must be called before any other function of the package. If it is not called, the default DLL search order of Windows is used.

5.3.2 Init

[Function] `func Init() error`

[Purpose] This function is used to initialize resources. It must be called once before any other SDK function.

[Parameter Description] None

[Return Value]

- `nil`: Succeeded. The C result 1 (already initialized) is also reported as `nil`.
- `error`: Failed (See Appendix 2.)

5.3.3 Terminate

[Function] `func Terminate() error`

[Purpose] This function is used to release resources. Close all `*Device` and `*DB` values before calling it.

[Parameter Description] None

[Return Value]

- `nil`: Succeeded
- `error`: Failed (See Appendix 2.)

5.3.4 GetDeviceCount

[Function] `func GetDeviceCount() (int, error)`

[Purpose] This function is used to acquire the number of devices.

[Parameter Description] None

[Return Value]

- `n, nil`: Device count (`n >= 0`)
- `0, error`: The function fails to be called (See Appendix 2.)

5.3.5 OpenDevice

[Function] `func OpenDevice(index int) (*Device, error)`

[Purpose] This function is used to start a device.

[Parameter Description] `index` - Device index (starting from 0)

[Return Value]

- `*Device, nil`: Device operation instance
- `nil, error`: Failed. The C function reports failure with a null handle, so the binding returns `ErrOpenDevice (-6)`.

5.3.6 Device.Close

[Function] `func (d *Device) Close() error`

[Purpose] This function is used to shut down a device. Calling it again on a closed device returns `nil`.

[Parameter Description] None

[Return Value]

- `nil`: Succeeded
- `error`: Failed (See Appendix 2.)

5.3.7 Device.SetParameter

[Function] `func (d *Device) SetParameter(code int, value []byte) error`

[Purpose] This function is used to set fingerprint reader parameters.

[Parameter Description]

- `code`: Parameter code (For details, see Appendix 1.)
- `value`: Parameter value. The length of the slice is passed as `cbParamValue`.

[Return Value]

- `nil`: Succeeded
- `error`: Failed (See Appendix 2.)

[Helpers] `SetParameterInt(code, v int32) error` encodes a 4-byte integer. `SetFormat(f Format) error`, `SetLED(color LED, on bool) error` and `Buzz(on bool) error` are short-cuts for the corresponding codes.

5.3.8 Device.GetParameter

[Function] func (d *Device) GetParameter(code int, size int) ([]byte, error)

[Purpose] This function is used to acquire fingerprint reader parameters.

[Parameter Description]

- code: Parameter code
- size: Size of the buffer to allocate for the value, based on the parameter code (4 for an Int, 4 for the VID/PID array, a larger buffer such as 64 for strings)

[Return Value]

- value, nil: The returned parameter value, already cut to the size reported by the SDK (cbParamValue [out])
- nil, error: Failed (See Appendix 2.)

[Helpers] GetParameterInt(code int) (int32, error), ImageWidth() (int, error), ImageHeight() (int, error), ImageSize() (int, error), VIDPID() (vid, pid uint16, err error), Vendor() (string, error), ProductName() (string, error) and SerialNumber() (string, error).

5.3.9 Device.AcquireFingerprint

[Function] func (d *Device) AcquireFingerprint(image []byte) ([]byte, error)

[Purpose] This function is used to capture a template (and the fingerprint image).

[Parameter Description]

- image [out]: Buffer that receives the fingerprint image. Its length is passed as cbFPIImage, and must be at least width*height bytes (use ImageSize).

[Return Value]

- template, nil: The captured template, trimmed to the actual size (the binding pre-allocates MaxTemplateSize = 2048 bytes)
- nil, error: Failed (See Appendix 2.)

[Note] The call returns ErrCaptureFailed (-8) when no finger is on the sensor. Applications normally poll in a loop and ignore this error until a finger is detected.

5.3.10 Device.AcquireFingerprintImage

[Function] func (d *Device) AcquireFingerprintImage(image []byte) error

[Purpose] This function is used to capture an image only.

[Parameter Description] image [out] - Buffer that receives the fingerprint image (length passed as cbFPIImage)

[Return Value]

- nil: Succeeded
- error: Failed (See Appendix 2.)

5.3.11 NewDB

[Function] func NewDB() (*DB, error)

[Purpose] This function is used to create an algorithm cache.

[Parameter Description] None

[Return Value]

- *DB, nil: Cache instance
- nil, error: Failed (the C function returned a null handle)

5.3.12 DB.Close

[Function] func (db *DB) Close() error

[Purpose] This function is used to release an algorithm cache. It is idempotent.

[Parameter Description] None

[Return Value]

- nil: Succeeded
- error: Failed (See Appendix 2.)

5.3.13 DB.Merge

[Function] func (db *DB) Merge(t1, t2, t3 []byte) ([]byte, error)

[Purpose] This function is used to combine three pre-registered fingerprint templates into one registered template.

[Parameter Description] t1, t2, t3 - Pre-registered fingerprint templates 1, 2, and 3 (three captures of the same finger)

[Return Value]

- regTemplate, nil: The registered template, trimmed to the actual size (the binding pre-allocates 2048 bytes)
- nil, error: Failed, for example ErrMergeFailed (-22)

5.3.14 DB.Add

[Function] func (db *DB) Add(fid uint32, template []byte) error

[Purpose] This function is used to add a registered fingerprint template to the cache.

[Parameter Description]

- fid: Fingerprint ID (32-bit unsigned integer larger than 0)
- template: Registered template (as returned by Merge)

[Return Value]

- nil: Succeeded
- error: Failed (See Appendix 2.)

5.3.15 DB.Delete

[Function] func (db *DB) Delete(fid uint32) error

[Purpose] This function is used to delete the registered template of a specified ID.

[Parameter Description] fid - Fingerprint ID

[Return Value]

- nil: Succeeded
- error: Failed (See Appendix 2.)

5.3.16 DB.Clear

[Function] func (db *DB) Clear() error

[Purpose] This function is used to clear the cache.

[Parameter Description] None

[Return Value]

- nil: Succeeded
- error: Failed (See Appendix 2.)

5.3.17 DB.Count

[Function] func (db *DB) Count() (uint32, error)

[Purpose] This function is used to acquire the number of fingerprint templates in the cache.

[Parameter Description] None

[Return Value]

- count, nil: Fingerprint count
- 0, error: Failed (See Appendix 2.)

5.3.18 DB.Identify

[Function] func (db *DB) Identify(template []byte) (fid, score uint32, err error)

[Purpose] This function is used to conduct 1:N comparison against all templates in the cache.

[Parameter Description] template - Fingerprint template to look for (the length is passed as cbTemplate)

[Return Value]

- fid, score, nil: The matching fingerprint ID and the comparison score
- 0, 0, error: Failed, for example ErrMatchFailed (-20) when no template matches (See Appendix 2.)

5.3.19 DB.Match

[Function] func (db *DB) Match(t1, t2 []byte) (int, error)

[Purpose] This function is used to compare whether two fingerprint templates match (1:1 comparison).

[Parameter Description] t1, t2 - The two templates (their lengths are passed as cbTemplate1 and cbTemplate2)

[Return Value]

- score, nil: Comparison score (score >= 0). The caller decides whether the score is high enough to be accepted as a match.
- 0, error: The C function returned a negative value, which is converted to an Error (See Appendix 2.)

5.3.20 DB.ExtractFromImage

[Function] func (db *DB) ExtractFromImage(path string, dpi uint32) ([]byte, error)

[Purpose] This function is used to extract a fingerprint template from a BMP or JPG file.

[Parameter Description]

- path: Full path of a file

- dpi: Image DPI

[Return Value]

- template, nil: The extracted template, trimmed to the actual size
- nil, error: Failed (See Appendix 2.)

[Note] Only the SDK of the standard version supports this function. Other versions return `ErrNotSupported` (-4).

6 Usage Example

The typical flow is: initialize, open a reader, capture three times, merge into a registered template, add it to the cache, then identify later captures.

```
if err := zkfp.Init(); err != nil { log.Fatal(err) }
defer zkfp.Terminate()

dev, err := zkfp.OpenDevice(0)
if err != nil { log.Fatal(err) }
defer dev.Close()

db, err := zkfp.NewDB()
if err != nil { log.Fatal(err) }
defer db.Close()

size, _ := dev.ImageSize()
img := make([]byte, size)

capture := func() []byte {
    for {
        tpl, err := dev.AcquireFingerprint(img)
        if err == nil { return tpl }
        if !errors.Is(err, zkfp.ErrCaptureFailed) { log.Fatal(err) }
        time.Sleep(100 * time.Millisecond) // no finger yet
    }
}

// Registration: three captures of the same finger
t1, t2, t3 := capture(), capture(), capture()
reg, err := db.Merge(t1, t2, t3)
if err != nil { log.Fatal(err) }
if err := db.Add(1, reg); err != nil { log.Fatal(err) }

// Identification (1:N)
fid, score, err := db.Identify(capture())
if err != nil { log.Fatal(err) }
fmt.Println("matched id", fid, "score", score)
```

7 Appendixes

7.1 Appendix 1: List of Common Parameter Codes

Integer values are passed as 4-byte little-endian values. Strings are returned as NUL-terminated byte strings, and the string helpers trim the terminator.

7.2 Appendix 2: Descriptions of Returned Error Values

Every error is an `zkfp.Error`. Test for a specific one with `errors.Is(err, zkfp.ErrNoDevice)`, or read the raw code with `errors.As`. Any other code is returned as an `Error` with that numeric value and the message “unknown error”.

7.3 Appendix 3: Notes for Implementers

- The binding uses `golang.org/x/sys/windows` instead of the frozen standard `syscall` package. Each `ZKFPM_*` symbol is resolved lazily with `windows.NewLazyDLL("libzkfp.dll").NewProc()`. Calls go through `Proc.Call`, so no cgo is needed. The dependency is fetched automatically by `go get`.
- `windows.NewLazySystemDLL` is **not** used, because it only searches the Windows system directory, where `libzkfp.dll` is normally not installed. When `SetDLLPath` is given a full path, the DLL is loaded from exactly that path, which also avoids DLL preloading (search-order hijacking).
- Handles returned by the DLL are stored as `uintptr`. A zero handle means failure.
- Go slices passed to the DLL must stay referenced until the call returns (`runtime.KeepAlive`), and the DLL must not keep the pointers after the call returns.
- `ZKFPM_GetParameters` and the other [in/out] size arguments are implemented by passing a pointer to a `uint32` that holds the buffer size on entry and the returned size on exit.

Code	Go constant	Property	Data Type	Description
1	ParamImageWidth	Read-only	Int	Image width
2	ParamImageHeight	Read-only	Int	Image height
3	ParamImageDPI	Read-write (LIVEID20R only)	Int	Image DPI (750/1000 recommended for children)
106	ParamImageSize	Read-only	Int	Image data size
1015	ParamVIDPID	Read-only	4-byte array	VID & PID bytes (former two indicate VID, latter two PID)
2002	ParamAntiFake	Read-write (LIVEID20R only)	Int	Anti-fake function (1: enable; 0: disable)
2004	ParamAntiFakeStatus	Read-only	Int	True if lower five bits are all 1's (value&31==31)
1101	ParamVendor	Read-only	String	Vendor information
1102	ParamProductName	Read-only	String	Product name
1103	ParamSerialNumber	Read-only	String	Device SN
101	ParamLEDWhite	Write-only	Int	1 indicates white light blinks; 0 indicates disabled
102	ParamLEDGreen	Write-only	Int	1 indicates green light blinks; 0 indicates disabled
103	ParamLEDRed	Write-only	Int	1 indicates red light blinks; 0 indicates disabled
104	ParamBuzzer	Write-only	Int	1 indicates buzzing started; 0 indicates disabled
10001	ParamFormat	Write-only (ISO/ANSI only)	Int	0: ANSI378

Code	Go value	Description
0	(nil)	Operation succeeded
1	(nil)	Initialized (already initialized; <code>Init</code> treats it as success)
-1	<code>ErrInitLib</code>	Failed to initialize the algorithm library
-2	<code>ErrInitCapture</code>	Failed to initialize the capture library
-3	<code>ErrNoDevice</code>	No device connected
-4	<code>ErrNotSupported</code>	Not supported by the interface
-5	<code>ErrInvalidParam</code>	Invalid parameter
-6	<code>ErrOpenDevice</code>	Failed to start the device
-7	<code>ErrInvalidHandle</code>	Invalid handle
-8	<code>ErrCaptureFailed</code>	Failed to capture the image
-9	<code>ErrExtractFailed</code>	Failed to extract the fingerprint template
-10	<code>ErrAbort</code>	Suspension operation
-11	<code>ErrNoMemory</code>	Insufficient memory
-12	<code>ErrBusy</code>	The fingerprint is being captured (the device is busy)
-13	<code>ErrAddFailed</code>	Failed to add the fingerprint template to the memory
-14	<code>ErrDeleteFailed</code>	Failed to delete the fingerprint template
-17	<code>ErrOperationFailed</code>	Operation failed (other error)
-18	<code>ErrCaptureCancelled</code>	Capture cancelled
-20	<code>ErrMatchFailed</code>	Fingerprint comparison failed
-22	<code>ErrMergeFailed</code>	Failed to combine registered fingerprint templates
-23	<code>ErrOpenFile</code>	Opening the file failed
-24	<code>ErrImageProcess</code>	Image processing failed